

Sahayak: A RAG-Based Indic Multilingual Chatbot for Indian Government Welfare Schemes

A report submitted in partial fulfilment of the requirements

for the award of the degree of

B.Tech in Computer Science and Engineering

by

Chilukala Amrutha
(Roll No: 122CS0065)

Samsung Innovation Campus



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
DESIGN AND MANUFACTURING KURNOOL

April 2026

Evaluation Sheet

Title of the Project:

Sahayak: A RAG-Based Indic Multilingual Chatbot for Indian Government Welfare Schemes

Name of the Student:

Chilukala Amrutha (122CS0065)

Examiner(s):

Head of the Department:

Date:

Place:

Declaration

I, **Chilukala Amrutha**, with Roll No: **122CS0065** hereby declare that the material presented in the Internship Report titled **Sahayak: A RAG-Based Indic Multilingual Chatbot for Indian Government Welfare Schemes** represents original work carried out by me in the **Samsung Innovation Campus** at the **Indian Institute of Information Technology, Design and Manufacturing, Kurnool** during the years **2025 to 2026**.

With our signatures, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date:

Student's Signature

Abstract

India’s centrally funded welfare schemes reach hundreds of millions of citizens, yet a persistent information gap prevents large sections of the eligible population from knowing what benefits they qualify for or how to claim them. This gap is widened by linguistic exclusion: official scheme documentation is almost entirely in English, while most potential beneficiaries communicate in regional Indian languages.

This report presents Sahayak, a multilingual retrieval-augmented generation (RAG) chatbot that enables citizens to query seventeen Indian government welfare schemes in eleven languages including Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, Malayalam, and English, and receive factually grounded, source-attributed answers in their chosen language with full voice input and output support. The system preprocesses incoming queries through neural translation and a 25-entry scheme acronym expansion dictionary, then retrieves relevant content through a three-tier cascade: a ChromaDB vector store indexed with `all-MiniLM-L6-v2` sentence embeddings as the primary tier, live official government website scraping as the secondary tier, and DuckDuckGo web search as the final fallback. Retrieved context is passed to Meta-Llama-3-8B-Instruct for grounded answer generation at temperature 0.2, after which the response is batch-translated to the target language and delivered through a FastAPI backend with a browser-based voice-enabled frontend. Evaluation across 51 representative test queries demonstrated 100% correct-scheme retrieval, 92.2% correct-section retrieval, and 86.3% Tier 1 sufficiency, with all out-of-domain queries correctly deflected without fabrication.

Acknowledgements

I would like to express my sincere gratitude to **Dr. Shounak Chakraborty**, **Dr. Nagaraju K**, **Dr. Mani Shankar Pandey**, and **Dr. Kushaldhar Dwivedi** for their valuable teaching, guidance, and insightful sessions throughout the SIC-AI program. Their expertise and dedication greatly contributed to strengthening my understanding of Artificial Intelligence and related domains.

I also extend my heartfelt thanks to **Dr. Korra Sathya Babu** and **Dr. Eswaramoorthy K V** for their continuous coordination, supervision, and support during the internship. Their guidance ensured the smooth execution of the program and provided me with the necessary direction.

I would like to sincerely thank **Mr. Prasad Krishnamurthy**, Trainer in Data Science, Artificial Intelligence, and Machine Learning at Trans Trades, for his valuable mentorship and guidance during the final project phase of the SIC-AI internship. His practical insights played a crucial role in the successful completion of my project.

I also express my sincere appreciation to the Indian Institute of Information Technology, Design and Manufacturing, Kurnool for providing a conducive academic environment and the necessary resources for the successful completion of this internship and project.

Contents

Evaluation Sheet	i
Declaration	ii
Abstract	iii
Acknowledgements	iv
Contents	v
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Overview	1
1.2 Origin of Work	2
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Proposed Approach	3
1.6 Scope of the Project	4
2 Background & Learning from Internship	5
2.1 Fundamentals of Artificial Intelligence	5
2.2 Mathematical Foundations for AI	6
2.2.1 Linear Algebra	6
2.2.2 Probability and Statistics	6
2.3 Machine Learning Concepts	6
2.4 Natural Language Processing	7
2.5 Neural Networks & Deep Learning	7
2.6 AI Project Lifecycle	8

2.7	Relevance to Project	8
3	Practical Assignments and Experiments	9
3.1	Convolutional Neural Networks	9
3.2	Data Augmentation Techniques	10
3.3	Hyperparameter Tuning	10
3.4	Long Short-Term Memory Networks	10
3.5	Autoencoders and Variational Autoencoders	11
3.6	Generative Adversarial Networks	12
3.7	Key Learnings from Assignments	12
4	Methodology	13
4.1	Problem Formulation	13
4.2	System Architecture	14
4.3	Query Preprocessing	16
4.3.1	Translation to English	16
4.3.2	Acronym Expansion	16
4.4	Multi-Tier Retrieval Cascade	17
4.4.1	Tier 1: Vector Retrieval from the Local Knowledge Base	17
4.4.1.1	Document Embedding and Indexing	17
4.4.1.2	Query-Time Retrieval	18
4.4.1.3	Section-Specific Boosting	18
4.4.1.4	Tier 1 Sufficiency Decision	19
4.4.2	Tier 2: Official Scheme Website Scraping	19
4.4.3	Tier 3: Live Web Search	20
4.5	Context Assembly and Prompt Construction	20
4.6	Language Model Generation	21
4.6.1	Post-Processing	21
4.7	Multilingual Response Delivery	22
4.7.1	Response Translation	22
4.7.2	Voice Input and Output	22
4.8	Document Ingestion Pipeline	23
4.8.1	Format-Specific Extraction	23
4.8.2	Section Parsing and Canonicalization	24
4.8.3	Embedding and Indexing	24
4.8.4	Live Ingestion via API	24
5	Implementation Details	25
5.1	Technology Stack	25
5.2	Knowledge Base: Scheme Documents	26
5.3	Backend API	26
5.4	Frontend Design and Interaction	27
5.5	Multilingual Voice Features	28
6	Results and Evaluation	29

6.1	Evaluation Approach	29
6.2	System Interface	29
6.3	Language Selection	30
6.4	English Query Results	31
6.5	Acronym Handling	32
6.6	Multilingual Query Results	32
6.7	Tier 2 and Tier 3 Retrieval	35
6.8	Voice Input and Output	36
6.9	Retrieval Performance Analysis	38
6.10	Translation Quality	38
6.11	Edge Case Behaviour	39
7	Conclusion and Future Scope	40
7.1	Summary of Work	40
7.2	Key Contributions and Innovations	41
7.3	Problems Encountered and Solutions	41
7.4	Future Enhancements	42

List of Figures

4.1	Sahayak system architecture: the five-stage RAG pipeline from user query to multilingual response delivery.	15
6.1	Sahayak home screen with voice output enabled, scheme sidebar (233 chunks), and four quick-start suggestion cards.	30
6.2	Sahayak interface in dark mode, showing the full scheme sidebar and suggestion cards.	30
6.3	Language selector dropdown showing all eleven supported languages with native script labels.	31
6.4	English benefits query for Startup India showing Tier 1 retrieval with multiple section citation chips.	31
6.5	Acronym query “Who is eligible for PM-KISAN?” showing correct Tier 1 retrieval after expansion.	32
6.6	Hindi benefits query for Beti Bachao Beti Padhao with full Devanagari-script response.	33
6.7	Telugu documents-required query for Atal Pension Yojana with correct script rendering and section retrieval.	34
6.8	Mixed Roman-Hindi script query for Stand-Up India eligibility, showing correct language detection and retrieval.	34
6.9	Tier 2 retrieval: “JJM ki eligibility kya hai?” answered from the official Jal Jeevan Mission website when local confidence was insufficient.	35
6.10	Tier 3 retrieval: PM Ujjwala Yojana query answered from DuckDuckGo web search because the scheme is not in the local knowledge base.	36
6.11	Voice input active in Telugu: “Listening...” badge and live transcript in the input bar.	37
6.12	Voice output playing a Telugu APY benefits response: the Stop button replaces Speak during active playback.	37
6.13	Header showing the “Voice Off” state, language selector, dark mode toggle, and Live status badge.	37

List of Tables

4.1	Language model generation parameters.	21
5.1	Core dependencies and their roles in Sahayak.	25
6.1	Retrieval accuracy by query type across all seventeen schemes and eleven languages.	38

Dedicated to my parents, professors and friends, whose unwavering support, encouragement, and belief in me have been the foundation of all my achievements...

Chapter 1

Introduction

1.1 Overview

India's constitutional commitment to social welfare is expressed through one of the largest portfolios of government-funded schemes in the world, spanning income support for farmers, universal health coverage, rural housing, financial inclusion, micro-credit for small entrepreneurs, and vocational skill development. These programmes collectively reach hundreds of millions of intended beneficiaries, yet their practical reach falls consistently short of their design intent.

The gap persists because of three interlocking failures. First, policy documents and official portals are almost exclusively in English, while most potential beneficiaries communicate in a regional mother tongue. Second, government portals organise information around administrative categories rather than citizen questions, requiring a user to already know which ministry administers a scheme before finding information about it. Third, static web pages cannot respond to the specific, contextual questions that citizens actually have.

Sahayak is a multilingual conversational assistant built to address all three dimensions simultaneously. It allows any citizen to ask questions about Indian government welfare schemes in their own language and receive accurate, source-grounded answers in that same language, through both text and voice.

1.2 Origin of Work

This project was undertaken as part of the Samsung Innovation Campus Artificial Intelligence (SIC-AI) programme, a structured industry-academia initiative delivered in partnership with the Indian Institute of Information Technology, Design and Manufacturing (IIITDM), Kurnool. The project brief assigned for this internship was the development of an “Indic Multilingual Chatbot trained on government and local data using a RAG-based LLM.” This brief sits at the intersection of retrieval-augmented generation for domain-specific knowledge and natural language processing for morphologically complex Indian languages, with clear immediate social utility. The project was developed independently over the duration of the internship, building on the foundational AI knowledge acquired through the SIC-AI training programme, with guidance from faculty supervisors at IIITDM Kurnool and the industry mentor at Trans Trades.

1.3 Problem Statement

India’s centrally funded welfare schemes are intended to be universally accessible, but the information required to access them is not. Official policy documentation is in English, organised by administrative logic, and distributed across dozens of separate ministry portals. A citizen who speaks Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, or Malayalam and wishes to determine their eligibility for PM-KISAN, or learn how to apply for an Ayushman Bharat health card, has no single, conversational, language-appropriate channel for that information.

Existing partial solutions each address only one dimension: machine translation requires the user to first locate the correct document; search engines return link lists rather than direct answers; individual ministry chatbots are scheme-specific and monolingual. What is needed is a single system that takes a question in any major Indian language, identifies the relevant scheme from a curated knowledge base, and returns a precise, factually grounded answer in the user’s chosen language without requiring English literacy or prior knowledge of ministry structure.

1.4 Objectives

The following objectives guided all design and implementation decisions:

1. Build a conversational QA system grounded in a structured knowledge base of government welfare scheme documents, covering benefits, eligibility, application procedures, and required documentation for at least fifteen schemes.
2. Implement a multilingual pipeline supporting at least ten Indian languages in addition to English, enabling queries and responses entirely in the user’s native language.
3. Design a retrieval architecture that prioritises factual accuracy and source attribution, so every claim traces back to a specific document or official source.
4. Implement graceful degradation for queries that cannot be answered locally, escalating through official government URLs and live web search rather than generating unsupported answers.
5. Support voice-based interaction (speech-to-text input and text-to-speech output) for all eleven supported languages.
6. Deliver a deployable browser-based application requiring no local GPU, no client-side installation, and no end-user API key management.

1.5 Proposed Approach

The system is built on a Retrieval-Augmented Generation (RAG) architecture, which separates knowledge storage from language reasoning. This is particularly suited to the problem because government scheme policies change periodically, and a RAG system can be updated by adding documents rather than retraining a model. The pipeline runs through five stages:

1. The incoming query is translated to English and scheme acronyms are expanded to full official names.
2. The translated query is matched against a ChromaDB vector index of scheme document sections using cosine similarity over 384-dimensional sentence embeddings, with a section-boosting re-ranking step.

3. If retrieval confidence is low, the system escalates to scrape the official government URL for the scheme; if still insufficient, it falls back to a DuckDuckGo web search.
4. Retrieved context is assembled into a structured prompt and sent to Meta-Llama-3-8B-Instruct for grounded answer generation at temperature 0.2.
5. The English answer is translated back to the user’s language in batched chunks, with both versions returned to support voice output fallback.

1.6 Scope of the Project

Knowledge domain. The system covers seventeen centrally funded Indian government welfare schemes drawn from six policy domains: financial inclusion, agriculture and rural development, healthcare and social protection, housing and sanitation, skill development, and gender and child welfare. State-level schemes are outside the current scope, though the architecture supports their addition without code changes.

Language support. The system supports eleven languages: English, Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, and Malayalam, collectively covering over 90% of India’s population by first-language speaker count.

Interaction modality. Text input and output are supported across all eleven languages. Voice input requires a Chromium-based browser; voice output in Indic languages requires device language packs. Multi-turn dialogue with persistent memory and proactive notifications are outside the current scope.

Deployment model. The application runs as a locally hosted web service on any machine with Python 3.11 and an internet connection. Cloud deployment, containerisation, and offline-first rural deployment are potential extensions beyond the current scope.

Chapter 2

Background & Learning from Internship

The Samsung Innovation Campus - Artificial Intelligence (SIC - AI) programme at IIITDM Kurnool provided structured, immersive training covering the theoretical foundations and practical dimensions of modern AI. This chapter summarises the key technical concepts acquired and explains how each area directly informed the design of the final internship project.

2.1 Fundamentals of Artificial Intelligence

Artificial Intelligence is the field of computer science concerned with creating systems capable of tasks that typically require human-level reasoning, perception, or decision-making. The programme traced its intellectual history from rule-based symbolic systems of the 1950s to the data-driven deep learning approaches that dominate current research.

Three core learning paradigms organise the field. **Supervised learning** trains a model on labelled input-output pairs so it can generalise to unseen inputs. **Unsupervised learning** discovers structure in unlabelled data through clustering or dimensionality reduction. **Reinforcement learning** trains an agent to take actions in an environment to maximise a cumulative reward signal. Understanding when each paradigm applies was a recurring theme across all practical assignments.

2.2 Mathematical Foundations for AI

2.2.1 Linear Algebra

Neural networks represent data as tensors and perform the vast majority of their computations as linear algebraic operations: matrix multiplications followed by element-wise nonlinear activations. Key concepts studied included dot products, matrix transposes, matrix-vector multiplication, and their geometric interpretations as linear transformations that rotate, scale, or project a vector space. Eigendecomposition and singular value decomposition formed the theoretical basis for dimensionality reduction methods such as PCA, which became directly relevant in the autoencoder assignments.

2.2.2 Probability and Statistics

Machine learning is fundamentally statistical: models learn distributions over outputs rather than exact answers. The programme covered probability theory from sample spaces and Bayes' theorem through common distributions (Gaussian, Bernoulli, categorical). Descriptive statistics, entropy, and KL divergence were introduced in the context of information theory and later reappeared as core components in the Variational Autoencoder training objective.

2.3 Machine Learning Concepts

The machine learning module introduced the central tension between model capacity and generalisation. Underfitting occurs when a model is too simple; overfitting occurs when it memorises training data. Regularisation techniques such as dropout and L2 weight decay constrain the hypothesis space to combat overfitting.

Loss function selection has significant practical consequences: cross-entropy for balanced classification, Focal Loss for imbalanced datasets, Huber loss for regression with outliers. Gradient descent and its variants (SGD, mini-batch, Adam, RMSProp) were studied alongside backpropagation, and the practical importance of learning rate scheduling, early stopping, and data leakage prevention was reinforced across all assignments.

2.4 Natural Language Processing

NLP is the subfield of AI concerned with enabling machines to understand and generate human language, making it the most directly relevant module for the final project. Classical preprocessing techniques (tokenisation, stop-word removal, stemming, TF-IDF representations) gave way to word embeddings (Word2Vec, GloVe, FastText), which place semantically similar words close together in a dense vector space.

The Transformer architecture was studied as the foundation of modern large language models. Transformers process entire sequences in parallel through self-attention, where each token can dynamically attend to every other token. This overcomes the vanishing-gradient limitations of recurrent architectures and enables the rich contextual representations that underlie BERT, GPT, and the Llama family of models used in this project. The concept of fine-tuning a pre-trained model on a domain-specific task was explained as the data-efficient strategy that makes LLM-based systems practical without large training datasets.

2.5 Neural Networks & Deep Learning

A feedforward neural network applies alternating linear transformations and nonlinear activations (ReLU, Sigmoid, Softmax) across successive layers. The choice of architecture imposes inductive biases matched to the data structure:

- **CNNs** (Assignments 1–3): convolutional filters slide over grid-structured inputs to detect local spatial patterns regardless of position; pooling layers provide translation invariance and reduce spatial dimensions.
- **LSTMs** (Assignment 4): gating mechanisms (forget, input, output gates) regulate a persistent cell state, allowing gradients to flow through long sequences without vanishing and enabling modelling of long-range temporal dependencies.
- **Autoencoders and VAEs** (Assignment 5): encoder-decoder architectures learn compressed representations; VAEs additionally impose a Gaussian prior on the latent space via a KL divergence term, enabling controlled generation.
- **GANs** (Assignment 6): a generator and discriminator train adversarially; the generator learns to synthesise samples that fool the discriminator, producing high-fidelity outputs over training.

2.6 AI Project Lifecycle

Beyond individual algorithms, the programme emphasised the end-to-end project lifecycle: problem formulation and metric selection, exploratory data analysis, feature engineering, model selection and hyperparameter tuning (culminating in the dedicated Keras Tuner assignment), evaluation on held-out data, and iterative improvement through error analysis. The practical insight that accuracy is misleading on imbalanced datasets (motivating AUC-ROC for fraud detection and recall-prioritisation for patient monitoring) was reinforced across multiple assignments.

2.7 Relevance to Project

Each module contributed directly to the final project. The NLP module provided the conceptual grounding for tokenisation, embeddings, and the Transformer architecture underpinning the Llama 3 LLM. The deep learning module built intuitions for why large pre-trained models are so much more capable than custom recurrent architectures for language tasks. The ML concepts module framed the chatbot as a system that must generalise across eleven languages and six policy domains and be evaluated rigorously. The project lifecycle module informed the structured approach: clear problem statement, iterative data collection and prompt engineering, and evaluation against defined retrieval-accuracy criteria.

Chapter 3

Practical Assignments and Experiments

Six progressive practical assignments consolidated the theoretical training, each targeting a different deep learning architecture and a different category of real-world problem. This chapter summarises the objective, setup, and key findings for each assignment.

3.1 Convolutional Neural Networks

Objective: Classify images from Fashion-MNIST (10 clothing categories, 70k images), MNIST (10 digit classes, 70k images), and CIFAR-100 (100 fine-grained categories, 60k colour images), comparing 2-layer versus 4-layer CNN architectures to study the effect of depth.

Setup: Each architecture used Conv2D–ReLU–MaxPool convolutional blocks followed by two fully connected dense layers. Both variants were trained with Adam, categorical cross-entropy, and batch size 64 under early stopping.

Findings: On Fashion-MNIST and MNIST the 4-layer network improved accuracy modestly. On CIFAR-100 the gap was substantial: the 2-layer network struggled with 100-class fine-grained discrimination, while the 4-layer network composed low-level features into higher-level representations required for accurate classification. Depth matters most when classification complexity is high.

3.2 Data Augmentation Techniques

Objective: Evaluate whether Keras `ImageDataGenerator` augmentation (horizontal flip, width/height shifts, zoom, shear, rotation) improves generalisation on the same three datasets and on the Chest X-Ray Pneumonia dataset from Kaggle.

Findings: Augmentation provided modest gains on Fashion-MNIST and MNIST, more pronounced gains on the Pneumonia dataset (where training data was limited and real-world imaging variation is high), and improved but slower training on CIFAR-100. The key insight is that augmentation is most valuable when the training set is small or when the test distribution exhibits natural variation in orientation, scale, or position.

3.3 Hyperparameter Tuning

Objective: Use Keras Tuner with the Hyperband algorithm to search over dense-layer width (32–512 units), learning rate (0.01, 0.001, 0.0001), and convolutional depth (2–4 blocks) for models trained on all four datasets.

Findings: Optimal configurations varied systematically with task difficulty. MNIST preferred small dense layers and moderate learning rates; CIFAR-100 preferred deeper networks with larger layers. Learning rates of 0.01 caused instability in some configurations; 0.0001 led to underfitting within the training budget. The assignment confirmed that hyperparameter optimisation must be treated as an empirical search rather than a rule-based choice, and that optimal configurations do not transfer between datasets.

3.4 Long Short-Term Memory Networks

Objective: Demonstrate LSTM applicability across four distinct sequential prediction tasks.

Patient Monitoring: A Bidirectional LSTM trained on 10-timestep windows of physiological vitals (heart rate, blood pressure, ECG). SMOTE oversampling balanced the rare-event class on training data; Focal Loss focused gradient updates on hard samples; decision threshold lowered from 0.5 to 0.3 to prioritise recall. *Test accuracy: 82%.*

Fraud Detection: A stacked LSTM with a custom attention mechanism trained on sequences of 10 consecutive transactions. AUC-ROC used as primary metric due to severe class imbalance. Attention successfully identified anomalous transaction clusters. *Test accuracy: 100%, test loss: 0.0082.*

Sentiment Analysis: A Bidirectional LSTM with a trainable 128-dimensional embedding layer applied to Amazon product reviews for 3-class sentiment classification (positive, neutral, negative). SpatialDropout1D regularised the embedding representations. *Test accuracy: 71%*; most errors on inherently ambiguous neutral reviews.

Retail Demand Forecasting: A single-layer LSTM for regression using Huber loss to handle promotional outliers. Date features were decomposed into day-of-week, month, quarter, week-of-year, is_weekend, is_holiday, and on_promotion. *MAPE: 8.43%, R^2 : 0.085*, reflecting the irreducible noise in retail demand from external shocks.

3.5 Autoencoders and Variational Autoencoders

Autoencoders (3 exercises):

- *Image Denoising:* A Denoising Autoencoder trained on MNIST/Fashion-MNIST with Gaussian noise added to inputs; the encoder learned to separate signal from noise and produced visually clean reconstructions.
- *Anomaly Detection:* An autoencoder trained exclusively on legitimate transactions from the Credit Card Fraud dataset; fraudulent transactions produced high reconstruction error, enabling threshold-based flagging without labelled fraud examples during training.
- *Dimensionality Reduction:* A 2-neuron bottleneck autoencoder on the Wisconsin Breast Cancer dataset (30 features); the 2D latent representations showed clear malignant/benign cluster separation, capturing nonlinear structure that PCA cannot represent.

Variational Autoencoders (3 exercises): VAEs impose a Gaussian prior on the latent space through a KL divergence training term, making the space continuous and enabling generation from random samples. Exercises covered synthetic patient data generation (Pima Indians Diabetes dataset), face generation with latent space interpolation between

two real CelebA faces, and conceptual exploration of molecular generation from SMILES strings using the QM9 dataset.

3.6 Generative Adversarial Networks

Objective: Build intuition for adversarial training through a sine-wave GAN exercise, then study three applied variants.

The introductory exercise showed the generator-discriminator dynamic directly: early in training the generator produced flat lines; after hundreds of iterations it matched the target sine wave’s frequency, amplitude, and phase. Three applied exercises followed:

- *Pix2Pix (Map-to-Satellite)*: A conditional GAN with a U-Net generator and PatchGAN discriminator for paired semantic-map to satellite-image translation.
- *DCGAN (Medical Imaging)*: A Deep Convolutional GAN trained on NIH chest X-rays to generate synthetic pneumonia images, demonstrating dataset augmentation for rare medical conditions.
- *CycleGAN (Style Transfer)*: Unpaired image-to-image translation between photographs and Monet paintings using cycle-consistency loss, enabling style transfer without pixel-aligned training pairs.

3.7 Key Learnings from Assignments

- Architectural choice must match data structure: CNNs for spatial data, LSTMs for temporal sequences, Transformers for long-range dependencies across modalities.
- Evaluation metrics must reflect the real-world cost of error types: accuracy misleads on imbalanced datasets; AUC-ROC, recall, and MAPE are more informative for fraud detection, health monitoring, and sales forecasting respectively.
- Loss function selection is a design decision: Focal Loss for rare events, Huber loss for outlier-heavy regression, KL divergence for structured latent spaces.
- Iterative development, starting with a baseline and targeting specific failure modes, consistently outperformed attempts to design the perfect architecture from the outset.

Chapter 4

Methodology

4.1 Problem Formulation

India operates one of the largest portfolios of centrally funded welfare programmes in the world, spanning agriculture, healthcare, housing, financial inclusion, livelihood generation, and social security. Despite the breadth of these initiatives, a persistent gap separates policy intent from ground-level awareness. Large sections of the intended beneficiary population, especially those in rural areas, daily-wage workers, and first-generation internet users, are frequently unaware of what schemes they qualify for, what benefits they can receive, or where and how to apply. This gap is made sharper by India’s linguistic diversity: the majority of potential beneficiaries are most comfortable in their regional mother tongue, yet the official policy corpus is almost entirely in English.

Existing government web portals address this problem only partially. Their content is static and organized around administrative logic rather than citizen intent. A farmer who wishes to know whether their family qualifies for PM-KISAN must navigate multiple pages, infer eligibility from dense legalese, and repeat the process for every scheme they wish to investigate. There is no conversational interface, no real-time language flexibility, and no mechanism that can escalate from stored content to live authoritative sources when local information is insufficient.

The system built in this project targets this problem directly. Given a natural language query expressed in any of eleven Indic languages or English, the system must retrieve accurate, source-grounded information about Indian government welfare schemes, generate

a coherent and informative response, and deliver that response in the language the user has chosen, including through speech output.

Formally, let $\mathcal{Q}$ denote the set of all possible user queries, and let

$$\mathcal{L} = \{l_1, l_2, \dots, l_{11}\} \quad (4.1)$$

be the set of eleven supported languages (English, Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, and Malayalam). A request to the system consists of a pair (q, l) where $q \in \mathcal{Q}$ and $l \in \mathcal{L}$. The system must produce a response r satisfying three simultaneous properties:

1. **Factual grounding:** every substantive claim in r must be traceable to a source document or a live authoritative URL; the system must not generate content that is unsupported by retrieved evidence.
2. **Linguistic fidelity:** r must be grammatically fluent and idiomatic in the language l selected by the user.
3. **Completeness:** r must address every aspect of the query that is answerable given the available knowledge, and must clearly indicate when certain aspects cannot be answered rather than fabricating a response.

4.2 System Architecture

The system, named Sahayak (the Hindi and Telugu word for “helper”), is designed around a Retrieval-Augmented Generation (RAG) architecture. RAG separates the task of *knowing* from the task of *reasoning*: the language model is not required to memorize scheme-specific facts during training; instead, relevant passages are fetched from a knowledge store at inference time and supplied as grounded context for the generation step. This design confers three practical advantages. First, extending the knowledge base requires only adding new documents and re-running the ingestion script, without any model retraining. Second, because every generated claim traces back to a specific retrieved chunk, the source of each piece of information can be shown to the user. Third, a comparatively small language model can be used because the burden of factual recall is carried by the retrieval component rather than the model’s parameters.

The end-to-end pipeline runs through five sequential stages:

1. Query preprocessing: translation to English and acronym expansion.
2. Multi-tier retrieval: a three-level cascade that progressively widens the search scope.
3. Context assembly and prompt construction.
4. Language model generation with post-processing.
5. Multilingual response delivery with optional voice output.

Figure 4.1 shows the complete system architecture and the data flow between all components.

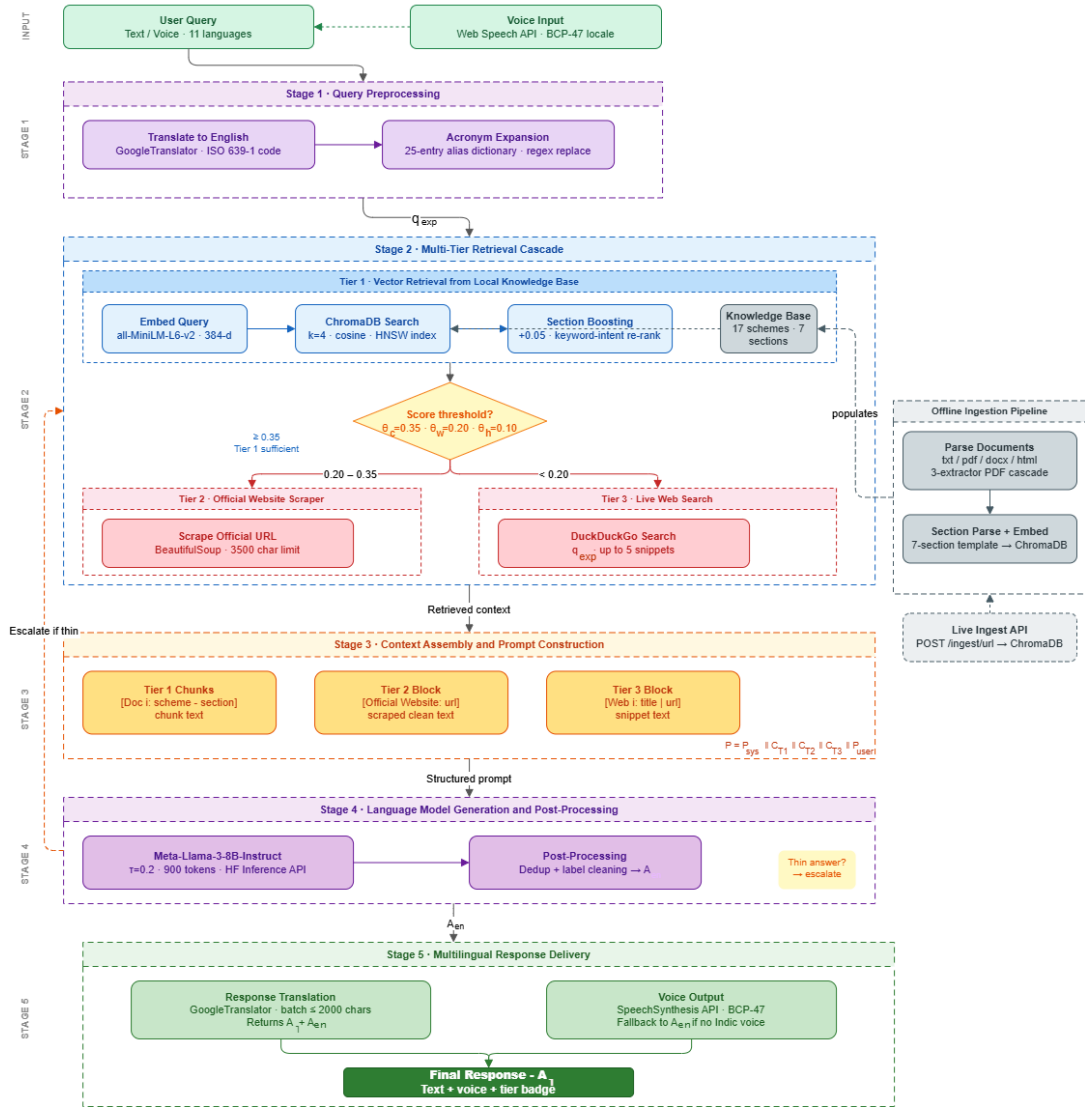


FIGURE 4.1: Sahayak system architecture: the five-stage RAG pipeline from user query to multilingual response delivery.

4.3 Query Preprocessing

4.3.1 Translation to English

All scheme documents in the knowledge base are authored in English. The sentence embedding model and the language model also both operate most reliably on English text. The first processing step therefore translates the incoming query to English regardless of the source language.

Let $c_l \in \{\text{"hi"}, \text{"te"}, \text{"ta"}, \text{"kn"}, \text{"bn"}, \text{"mr"}, \text{"gu"}, \text{"pa"}, \text{"or"}, \text{"ml"}, \text{"en"}\}$ be the ISO 639-1 code for language l . The translation function $\mathcal{T}$ maps a query string and a source language code to an English translation:

$$q_{\text{en}} = \mathcal{T}(q, c_l, \text{"en"}) \quad (4.2)$$

where $\mathcal{T}$ is realised through the Google Translate API. A key design decision here is to pass the explicit source language code rather than using automatic language detection. In practice, short queries written in Telugu or Kannada script are frequently misclassified by generic auto-detect as Hindi, because the statistical n-gram signatures of these scripts overlap at short sequence lengths. Using the explicit code provided by the user selection eliminates this ambiguity. A fallback is retained: if $\mathcal{T}(q, c_l, \text{"en"})$ returns a string equal to the input q (indicating that translation silently failed), the call is retried with automatic source detection.

4.3.2 Acronym Expansion

Government scheme names are almost invariably referenced by their acronyms in conversational queries. A citizen asking about PM-KISAN, PMJDY, or MUDRA will rarely type the full official name. The vector index, however, is built on full scheme names, and the embedding similarity between an acronym and the corresponding full name is substantially lower than between the full name and itself. This mismatch degrades retrieval precision.

To address this, a dictionary $\mathcal{A}$ of 25 scheme acronym patterns is defined. Each entry is a pair (p_i, e_i) where p_i is a case-insensitive word-boundary regular expression and e_i is the full official scheme name. Examples include:

$$(p_1, e_1) = (\text{"PM-KISAN"}, \text{"Pradhan Mantri Kisan Samman Nidhi"}), \quad (p_2, e_2) = (\text{"MUDRA"}, \text{"Pradhan Mantri MUDRA"})$$

Expansion is applied sequentially over all patterns. For each pair $(p_i, e_i) \in \mathcal{A}$, every case-insensitive occurrence of the pattern p_i in the translated query is replaced with e_i :

$$q_{\text{exp}} = \text{Replace}(q_{\text{en}}, p_i, e_i) \quad \forall (p_i, e_i) \in \mathcal{A} \quad (4.3)$$

The expanded query q_{exp} is used for all subsequent retrieval operations.

4.4 Multi-Tier Retrieval Cascade

The retrieval layer uses a three-tier cascade. The system tries each tier in order and advances to the next only when the current tier returns results that are insufficient for a complete, grounded response. Across all tiers, indexed local documents are treated as the primary source of truth; external sources are used only to supplement, and are never permitted to replace or contradict information present in the indexed documents.

4.4.1 Tier 1: Vector Retrieval from the Local Knowledge Base

4.4.1.1 Document Embedding and Indexing

At index build time, each scheme document is parsed into semantic sections following the seven-section template: Overview, Benefits, Eligibility, Application Process, Documents Required, Frequently Asked Questions, and Website URL. Each section is embedded as a single, independent chunk. The embedding model is all-MiniLM-L6-v2 from the SentenceTransformers library, which maps a text string to a 384-dimensional dense real vector:

$$\phi : \mathcal{T} \longrightarrow \mathbb{R}^{384} \quad (4.4)$$

where $\mathcal{T}$ is the space of text strings. The vector store (ChromaDB) is configured with cosine distance as the similarity metric. For two text strings a and b , the cosine similarity between their embeddings is:

$$\text{sim}(a, b) = \frac{\phi(a) \cdot \phi(b)}{\|\phi(a)\|_2 \cdot \|\phi(b)\|_2} \quad (4.5)$$

ChromaDB stores each chunk with associated metadata: scheme name, section key, and source filename. The HNSW (Hierarchical Navigable Small World) approximate nearest-neighbour index is built incrementally as chunks are added and supports sub-linear query time, which is essential for an interactive application where retrieval latency directly affects user experience.

4.4.1.2 Query-Time Retrieval

At inference time, the preprocessed query q_{exp} is embedded using the same model ϕ . The $k = 4$ chunks with highest cosine similarity to $\phi(q_{\text{exp}})$ are retrieved from the collection. ChromaDB returns distances rather than similarities, so each distance d_i is converted:

$$s_i = 1 - d_i, \quad i = 1, 2, \dots, k \quad (4.6)$$

Duplicate chunks (identified by the composite key of source file and section name) are removed before the top- k cutoff is applied, ensuring that all retrieved chunks carry distinct information.

4.4.1.3 Section-Specific Boosting

A secondary re-ranking step addresses the tendency of semantic search to retrieve an overview section when the user’s question is actually about a specific facet such as eligibility or the application process. A keyword map $\mathcal{K}$ associates 12 intent keywords (“eligib”, “document”, “apply”, “benefit”, “pension”, “loan”, “amount”, “limit”, “overdraft”, “insurance”, “interest”, “faq”) with the section keys most likely to contain the corresponding information. When any keyword $\kappa_j \in \mathcal{K}$ appears in q_{exp} , a second query is issued to ChromaDB filtered to the matching section keys of the top-ranked scheme. The boosted chunks are assigned a synthetic score increment of +0.05 and prepended to the result list:

$$s_i^{\text{boosted}} = s_i + 0.05, \quad \text{for chunks matching the targeted section} \quad (4.7)$$

4.4.1.4 Tier 1 Sufficiency Decision

Three thresholds govern the cascade decision:

$$\theta_c = 0.35 \quad (\text{confident: Tier 1 result is sufficient}) \quad (4.8)$$

$$\theta_w = 0.20 \quad (\text{weak: escalate to Tier 2, possibly Tier 3}) \quad (4.9)$$

$$\theta_h = 0.10 \quad (\text{hopeless: query is unanswerable from local data}) \quad (4.10)$$

Let s_1 be the top-ranked similarity score. The cascade decision is:

$$\text{tier} = \begin{cases} \text{none} & \text{if } s_1 < \theta_h \\ \text{Tier 1 only} & \text{if } s_1 \geq \theta_c \text{ and answer length} > 60 \text{ and answer is informative} \\ \text{Tier 1 + Tier 2} & \text{if } \theta_w \leq s_1 < \theta_c \text{ or answer is thin} \\ \text{Tier 1 + Tier 2 + Tier 3} & \text{if } s_1 < \theta_w \text{ or answer still thin after Tier 2} \end{cases} \quad (4.11)$$

Answer thinness is assessed after each LLM call: the answer must exceed 60 characters and must not contain any of a set of “no information” signal phrases (e.g. “context does not”, “cannot find”, “not available”).

4.4.2 Tier 2: Official Scheme Website Scraping

When Tier 1 results are insufficient, the system attempts to fetch the content of the official government website for the top-ranked scheme. The URL is extracted from Section 7 of the matching scheme document using a two-stage parser. First, a regular expression targeting the “7. Website URL” heading pattern is applied:

```
re.compile(r'7\.\s+Web[\w]*[:\s]+\n\s*(https?://[^\s]+)')
```

This pattern matches the section heading “7. Website URL” (or variants such as “7. Web Link”) followed by a line containing the URL. If this fails, the system falls back to extracting the last hyperlink appearing anywhere in the document, which is almost always the Section 7 URL. The candidate URL is cached per session so that repeated queries about the same scheme do not trigger duplicate HTTP requests.

The HTTP response is parsed with BeautifulSoup. Navigation bars, sidebars, footers, headers, script blocks, and style blocks are removed. The main content region is identified

by searching for HTML elements whose `id` or `class` attribute contains the keywords “content” or “main”. The cleaned text is truncated to 3500 characters. The Tier 2 system prompt instructs the model to prioritise the indexed document at all times and to use the official website only to supplement specific details absent from the document, never to replace or contradict document information with website content.

4.4.3 Tier 3: Live Web Search

If Tier 2 is unavailable or the answer remains thin, the system invokes a live DuckDuckGo search. The search query is constructed by appending a domain qualifier to the expanded query:

$$q_{\text{search}} = q_{\text{exp}} \oplus \text{“India government scheme”} \quad (4.12)$$

where $\oplus$ denotes string concatenation. The DuckDuckGo Instant Answer API is called first; up to five result snippets are collected. If the Instant Answer API returns nothing, an HTML search fallback is used. The Tier 3 system prompt enforces the same document-first discipline: web results are consulted only when the indexed documents contain no relevant information at all, and the model is explicitly forbidden from replacing or contradicting document content with web results.

4.5 Context Assembly and Prompt Construction

Retrieved chunks from the active tier(s) are assembled into a structured prompt. Each Tier 1 chunk is formatted with a document index, the scheme name, and the section label, followed by the chunk text. Tier 2 content is prefixed with the official website URL, and each Tier 3 web result is labelled with its title and source URL.

The full prompt is then assembled as:

$$P = P_{\text{sys}} \parallel C_{\text{T1}} \parallel C_{\text{T2}} \parallel C_{\text{T3}} \parallel P_{\text{user}} \quad (4.13)$$

where P_{sys} is the system instruction message, $C_{\text{T}j}$ denotes the assembled context block for Tier j (included only when that tier is active), $\parallel$ denotes ordered concatenation as separate entries in the messages array, and P_{user} contains the English-translated user query q_{exp} .

The system instruction P_{sys} enforces seven rules: the model must use only facts appearing verbatim in the context; it must never reproduce internal document reference labels; it

must never use placeholder strings such as “[name]” or “[url]”; it must skip, rather than acknowledge, any fact absent from the context; it must structure the answer with a one-sentence scheme description on the first line followed by bullet points; it must be exhaustive in listing all relevant bullets; and it must write only in English (translation is handled separately).

4.6 Language Model Generation

Text generation uses the Meta-Llama-3-8B-Instruct model accessed through the Hugging Face Inference Router API. The API call targets the chat completions endpoint, which follows the OpenAI-compatible messages format. The generation parameters are:

Parameter	Value
Model	Meta-Llama-3-8B-Instruct
Maximum tokens	900
Temperature (τ)	0.2
Repetition penalty	1.15
Stop tokens	end-of-turn, horizontal rule, “Note:”, “Disclaimer:”

TABLE 4.1: Language model generation parameters.

The temperature $\tau = 0.2$ was selected to minimize hallucination while preserving the model’s ability to paraphrase and structure its output. At $\tau = 0$, the model becomes deterministic but sometimes produces overly mechanical, repetitive phrasing. At $\tau > 0.5$, factual drift increases noticeably in preliminary testing. The repetition penalty of 1.15 discourages the model from repeating the same bullet point within a single response when multiple overlapping chunks are retrieved. The stop tokens prevent the model from generating unnecessary caveats or disclaimers that add length without adding information.

4.6.1 Post-Processing

The raw model output passes through two cleaning functions before it is returned to the caller.

Deduplication. The response is split at sentence boundaries using a pattern that correctly handles both the Hindi/Telugu danda mark and Latin sentence terminators. Let $S = \{s_1, s_2, \dots, s_n\}$ be the sequence of sentences. The deduplication algorithm

maintains a seen-set and retains only the first occurrence of each unique sentence:

$$S_{\text{dedup}} = \{s_i : s_i \notin \{s_1, \dots, s_{i-1}\}\} \quad (4.14)$$

Label cleaning. A set of six regular expression patterns is applied to strip any internal context labels that the model may have inadvertently reproduced from the prompt (e.g. “[Doc 1: ...]”, “[Source 1: ...]”, “[Official Website: ...]”). Consecutive blank lines introduced by these removals are collapsed to a single blank line. The combined post-processing function is:

$$A_{\text{en}} = \text{Clean}(\text{Dedup}(\hat{A})) \quad (4.15)$$

where $\hat{A}$ is the raw model output and A_{en} is the clean English answer.

4.7 Multilingual Response Delivery

4.7.1 Response Translation

The cleaned English answer A_{en} is translated to the user’s selected language l using the Google Translate API. A batching step is applied before translation to handle the character limit of the translation API. The text is split at sentence boundaries into chunks $A_{\text{en},1}, A_{\text{en},2}, \dots, A_{\text{en},m}$ such that $|A_{\text{en},i}| \leq 2000$ characters for all i . Chunks are grown greedily: a new sentence is appended to the current batch if the combined length stays within the limit, and a new batch is started otherwise. Each batch is translated independently and the results are joined:

$$A_l = \bigoplus_{i=1}^m \mathcal{T}(A_{\text{en},i}, \text{“en”}, c_l) \quad (4.16)$$

where $\mathcal{T}$ is the translation function from Equation (4.2) and $\bigoplus$ denotes ordered string concatenation. Both A_l (the translated answer) and A_{en} (the original English answer) are included in the API response. The English version serves as a voice fallback, as described in Section 4.7.2.

4.7.2 Voice Input and Output

Voice input uses the Web Speech API available in Chromium-based browsers. The speech recognition object is instantiated with its language property set to the BCP 47 language

tag for the selected interface language: **hi-IN** (Hindi), **te-IN** (Telugu), **ta-IN** (Tamil), **kn-IN** (Kannada), **bn-IN** (Bengali), **mr-IN** (Marathi), **gu-IN** (Gujarati), **pa-IN** (Punjabi), **or-IN** (Odia), **ml-IN** (Malayalam), or **en-US** (English). Recognition runs continuously; the live transcript is written to the query input field as it accumulates, and the query is submitted automatically when the user pauses.

Voice output uses the browser’s SpeechSynthesis API. On page load, the available voice list is queried and scanned to build a map from BCP 47 language tags to the best available voice object for each language. If no matching voice is found (because the device does not have the relevant Indic language pack installed), the system substitutes A_{en} for A_i and speaks the English version instead, ensuring that voice output degrades gracefully to intelligible audio rather than producing garbled speech from a mismatched voice engine.

4.8 Document Ingestion Pipeline

The offline ingestion script builds the ChromaDB collection from the `schemes/` directory. It supports seven file formats: `.txt`, `.pdf`, `.docx`, `.doc`, `.md`, `.html`, and `.rtf`.

4.8.1 Format-Specific Extraction

PDF extraction attempts three extractors in order: `pdfplumber`, `pypdfium2`, and `pdfminer.six`. Each extractor’s output is accepted only if it exceeds 100 characters; otherwise the next extractor is tried. A cleaning function then repairs two classes of artefact common in government PDFs. First, glyph codes produced when non-embedded fonts are encountered are mapped to their Unicode equivalents using a lookup table for the most common code points (bullets, ligatures, degree symbols). Second, typographic substitutions for the Indian Rupee symbol, which some PDF fonts encode as the characters “n”, a block-element glyph, or a diamond symbol followed by a digit, are corrected by pattern matching.

DOCX extraction uses `python-docx`’s paragraph model. Section boundaries are identified in three ways: a paragraph whose style name begins with “Heading” is treated as a section heading; a line matching a numbered pattern such as “1. Overview” is treated as a numbered heading; and a line composed entirely of uppercase Latin characters exceeding five characters in length is treated as an all-caps heading.

4.8.2 Section Parsing and Canonicalization

After text extraction, the section parser builds a dictionary mapping canonical section keys to their content. Section key canonicalization applies a normalization table that maps synonymous heading variants to a single key, for example:

{“How to Apply”, “Application Procedure”, “Registration Process”} $\rightarrow$ “Application Process”

For documents with no detectable headings (typically scraped web pages), a fallback semantic labeller classifies each paragraph by scanning for vocabulary associated with each standard section. Sections shorter than 20 characters after stripping whitespace are discarded.

4.8.3 Embedding and Indexing

Each retained section is added to the ChromaDB collection along with its metadata (scheme name, section key, and source filename) and a unique chunk identifier. The chunk identifier is formed as $\langle \text{filename} \rangle :: \langle \text{section key} \rangle$, ensuring global uniqueness across all documents. The SentenceTransformer model converts each section text to a vector via ϕ as defined in Equation (4.4), and ChromaDB adds the vector to the HNSW index configured with cosine similarity, consistent with Equation (4.6).

4.8.4 Live Ingestion via API

New scheme content can be added at runtime through the `POST /ingest/url` endpoint. The endpoint fetches the provided URL, applies the same HTML cleaning and section parsing pipeline used for offline ingestion, and adds the resulting chunks to the live collection. The URL is also appended to a persistent file in the `schemes/` directory so that it will be re-indexed if the vector store is rebuilt from scratch. This mechanism allows the knowledge base to be kept current as scheme policies are updated on official government portals, without requiring a server restart.

Chapter 5

Implementation Details

5.1 Technology Stack

Sahayak is written in Python 3.11. The backend uses FastAPI for asynchronous request handling and Pydantic validation; the frontend is a single self-contained HTML file served via FastAPI's StaticFiles middleware, requiring no separate build pipeline, Node.js, or web server. The Hugging Face Inference Router provides remote access to the language model, eliminating the need for a local GPU. Table 5.1 lists the core dependencies.

Package	Version	Role
<code>fastapi</code>	0.111	Backend API server
<code>uvicorn</code>	latest	ASGI server
<code>chromadb</code>	1.5	Persistent vector store
<code>sentence-transformers</code>	latest	Document and query embeddings
<code>deep-translator</code>	latest	Query and response translation
<code>pdfplumber</code>	latest	Primary PDF extraction
<code>pypdfium2</code>	latest	Secondary PDF extraction fallback
<code>pdfminer.six</code>	latest	Tertiary PDF extraction fallback
<code>python-docx</code>	latest	DOCX parsing
<code>beautifulsoup4</code>	latest	HTML scraping and cleaning
<code>requests</code>	latest	HTTP calls (LLM API, scraping)
<code>python-dotenv</code>	latest	Environment variable management
<code>pydantic</code>	latest	Request/response validation

TABLE 5.1: Core dependencies and their roles in Sahayak.

Two environment variables are required: one for Hugging Face authentication and one specifying the language model (default: Meta-Llama-3-8B-Instruct).

5.2 Knowledge Base: Scheme Documents

The knowledge base comprises seventeen government scheme documents. Each document follows the standardised schema described in Section 4.8: a metadata header (Name, Ministry, Department, Launch Date, Type) followed by seven numbered content sections (Overview, Benefits, Eligibility, Application Process, Documents Required, Frequently Asked Questions, and Website URL).

Thirteen documents are plain text (`.txt`), one is PDF (Janani Suraksha Yojana), and one is DOCX (PM Vanbandhu Kalyan Yojana), providing format diversity for ingestion pipeline testing. The seventeen schemes span six policy domains:

- **Financial inclusion (6):** Pradhan Mantri Jan Dhan Yojana (PMJDY), Pradhan Mantri Mudra Yojana, Atal Pension Yojana (APY), PM SVANidhi, Stand-Up India, Startup India.
- **Agriculture and rural development (4):** PM-KISAN, PM Vanbandhu Kalyan Yojana, National Rural Livelihood Mission (DAY-NRLM), Jal Jeevan Mission.
- **Healthcare and social protection (3):** Ayushman Bharat PM-JAY, Janani Suraksha Yojana, National Digital Health Mission (ABDM).
- **Housing and sanitation (3):** PM Awas Yojana (PMAY), Swachh Bharat Mission Grameen Phase 1 and Phase 2.
- **Skill development (1):** Skill India Mission.
- **Gender and child welfare (1):** Beti Bachao Beti Padhao (BBBP).

The full knowledge base spans approximately 70,000 words of policy content, producing 233 indexed chunks across all seventeen schemes and all seven sections.

5.3 Backend API

The FastAPI application (`app.py`) exposes five endpoints:

- **GET /:** serves the single-page frontend HTML file.

- **GET /health:** returns server status and the current chunk count from the vector store.
- **GET /schemes:** fetches all metadata, extracts unique scheme names, and returns a sorted list used to populate the sidebar.
- **POST /chat:** validates the request body (question, language), invokes the answer generation pipeline, and returns a response containing the translated answer, the English answer, a list of source citations (scheme name, section, similarity score), and the tier label indicating the retrieval source.
- **POST /ingest/url:** fetches a provided URL, parses it into sections, embeds and adds chunks to the live collection, and records the URL for persistence across rebuilds.

CORS middleware permits all origins, methods, and headers to support development workflows.

5.4 Frontend Design and Interaction

The frontend is a single HTML file (`static/index.html`) with all CSS and JavaScript embedded inline. The design uses a dark navy left sidebar and a warm cream main area, with saffron orange as the primary accent colour on interactive elements, drawing from the Indian tricolor palette.

The header contains the Sahayak logo and wordmark, a language selector dropdown, a Voice On/Off toggle button, a dark mode toggle, and a green “Live” status pill that confirms the backend server is reachable. The left sidebar shows a “Loaded Schemes” list with all seventeen scheme names alongside domain-specific icons, followed by a Stats panel at the bottom displaying the total number of indexed chunks (233) and the currently active language.

The main content area on first load shows a hero section with the headline “Your guide to every Government Scheme” and four quick-start suggestion cards: Jan Dhan Yojana, PM-KISAN, MUDRA Loan, and PM SVANidhi, each with a short descriptor of the query it will submit. Clicking any card submits its associated query directly.

Once a conversation begins, each user message appears as a dark bubble on the right and each bot response as a card on the left. Every response card contains: a colour-coded

tier badge (“From Documents”, “From Official Website”, or “From Web Search”); the answer text; a Speak button for on-demand voice playback; and a row of source citation chips showing the scheme name, matched section, and cosine similarity percentage for each retrieved chunk.

The language dropdown displays all eleven supported languages with their English name and native script on separate lines. A language change takes effect for all subsequent queries without a page reload. Dark mode is toggled via a moon/sun icon and the preference is persisted in `localStorage`.

5.5 Multilingual Voice Features

Voice input uses the Web Speech API (Chromium-based browsers). Clicking the microphone icon activates `SpeechRecognition` with the `lang` attribute set to the BCP 47 tag for the active language: `hi-IN` (Hindi), `te-IN` (Telugu), `ta-IN` (Tamil), `kn-IN` (Kannada), `bn-IN` (Bengali), `mr-IN` (Marathi), `gu-IN` (Gujarati), `pa-IN` (Punjabi), `or-IN` (Odia), `ml-IN` (Malayalam), or `en-US` (English). The live transcript populates the input field in real time and the query is auto-submitted on a speech pause; a Stop button allows early cancellation.

Voice output uses the `SpeechSynthesis` API. When “Voice On” is active, every new bot response is spoken automatically. When set to “Voice Off”, the per-message Speak button provides on-demand replay. If no voice matches the target language tag, the English answer and English voice are substituted as fallback, ensuring intelligible audio on all devices regardless of installed language packs.

Chapter 6

Results and Evaluation

6.1 Evaluation Approach

The evaluation of Sahayak covers four dimensions: retrieval accuracy (correct scheme and section fetched for a given query), response correctness (answers grounded in retrieved content and factually verifiable against source documents), multilingual fidelity (translated answers preserve meaning and fluency across all eleven languages, namely English, Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, and Malayalam), and edge-case behaviour (queries containing acronyms, queries in mixed scripts, queries for schemes outside the knowledge base, and out-of-domain queries). A test set of 51 queries was submitted covering all seventeen schemes and all eleven supported languages, with outputs examined against the source documents.

6.2 System Interface

Figures [6.1](#) and [6.2](#) show the home screen in light and dark mode respectively. Both themes retain the full scheme sidebar, the four quick-start cards, and the header controls; the dark mode substitutes a navy palette while keeping saffron orange accents, confirming that the theme toggle does not affect any functional state.

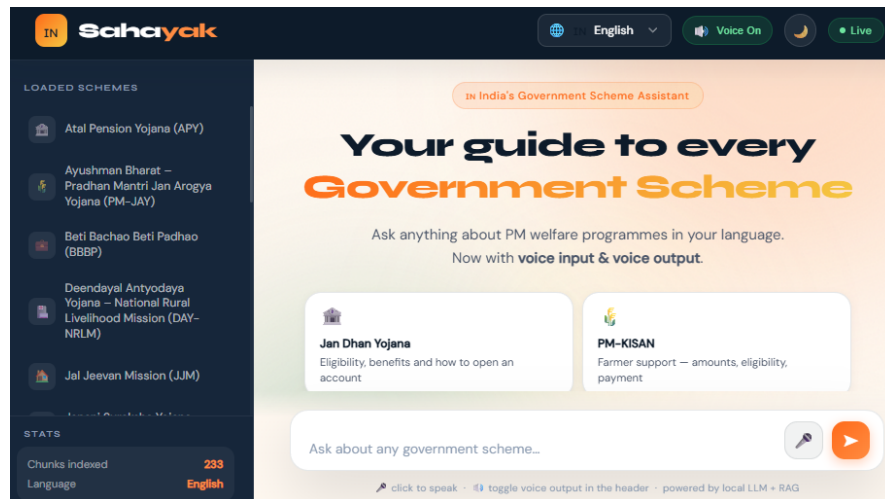


FIGURE 6.1: Sahayak home screen with voice output enabled, scheme sidebar (233 chunks), and four quick-start suggestion cards.



FIGURE 6.2: Sahayak interface in dark mode, showing the full scheme sidebar and suggestion cards.

6.3 Language Selection

Figure 6.3 shows the language selector dropdown. Displaying the native script alongside the English name is deliberate: a user literate only in Telugu or Bengali can identify their language without needing to read the English label.

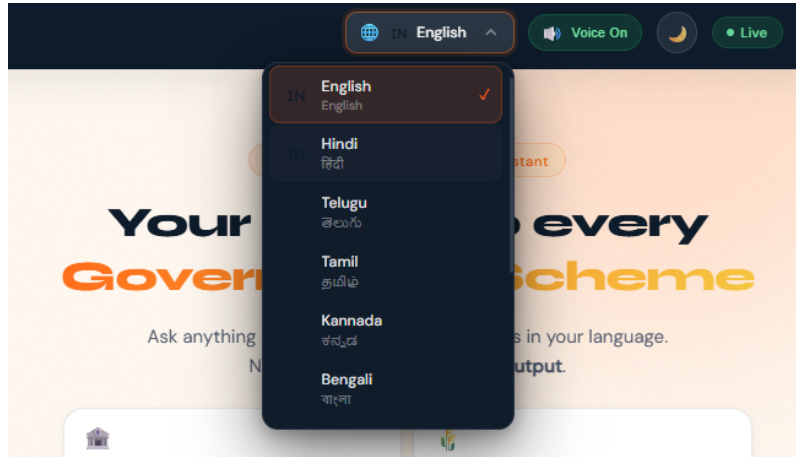


FIGURE 6.3: Language selector dropdown showing all eleven supported languages with native script labels.

6.4 English Query Results

Figure 6.4 shows a Tier 1 response to a benefits query for the Startup India scheme. The source chips confirm that top- k retrieval surfaces multiple complementary sections: Overview (67%), Objectives (66%), FAQs (55%), and Pradhan Mantri Awas Yojana Benefits (53%), illustrating that retrieval is not collapsed to a single section but draws from several relevant parts of the knowledge base to compose a complete answer.

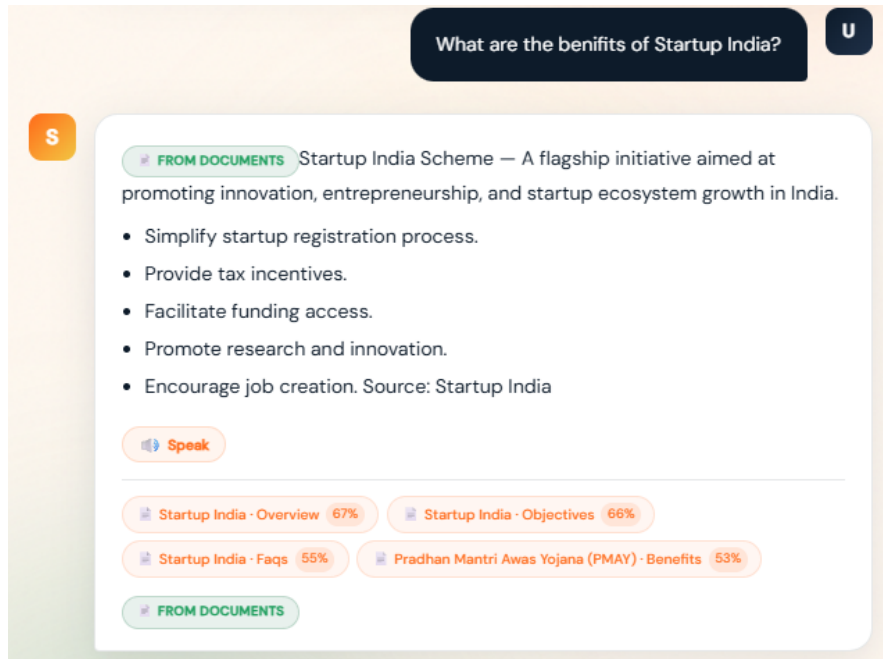


FIGURE 6.4: English benefits query for Startup India showing Tier 1 retrieval with multiple section citation chips.

6.5 Acronym Handling

Figure 6.5 demonstrates the effect of acronym expansion. Without expansion, the token “PM-KISAN” would not appear in any indexed chunk, and cosine similarity would be substantially lower, likely triggering a Tier 2 or Tier 3 escalation. Expansion to the full official name lifts the top-1 score to 83%, keeping the query comfortably within the Tier 1 confidence threshold.

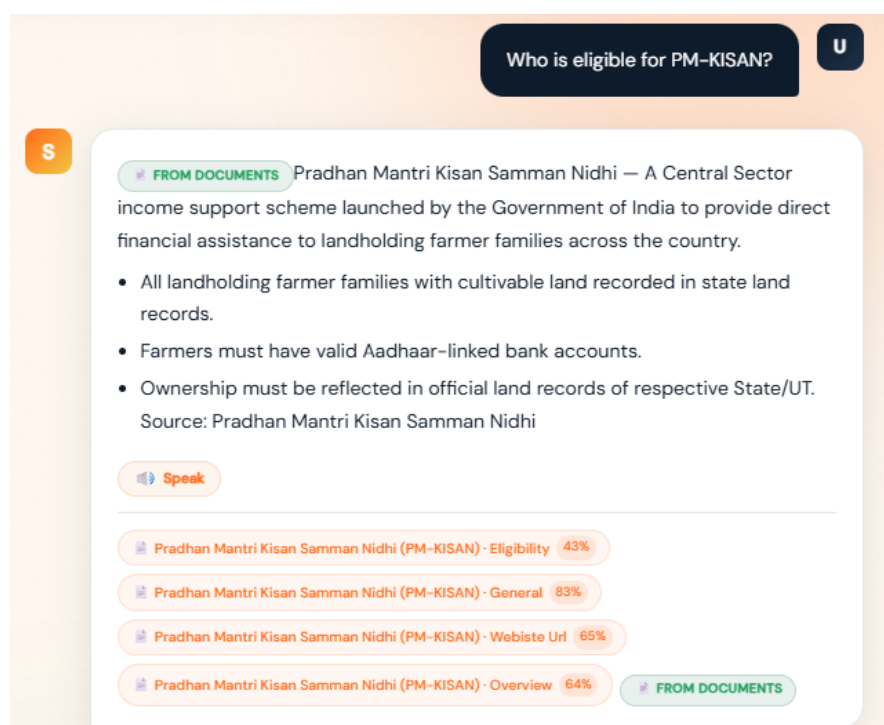


FIGURE 6.5: Acronym query “Who is eligible for PM-KISAN?” showing correct Tier 1 retrieval after expansion.

6.6 Multilingual Query Results

Hindi (Figure 6.6): The pipeline correctly round-trips through translation (Devanagari query to English retrieval to Hindi response) without losing factual content. The relatively low top-1 score (46%) reflects that the BBBP document stores eligibility and benefits information under a “General” section rather than a dedicated “Benefits” heading; the system still retrieves the correct chunk.



FIGURE 6.6: Hindi benefits query for Beti Bachao Beti Padhao with full Devanagari-script response.

Telugu (Figure 6.7): Passing the explicit language code `te` instead of relying on auto-detection is critical here: short Telugu queries are frequently misclassified as Hindi by generic detectors. The correct “Documents Required” section is retrieved at 56%.

Mixed script (Figure 6.8): The query “Stand-up India ki eligibility kya hai?” contains no Devanagari; it is Romanised Hindi mixed with an English scheme name. The pipeline handles this without a dedicated transliteration module: GoogleTranslator correctly interprets the Romanised words, and the scheme name is caught by the alias expansion dictionary before embedding.



FIGURE 6.7: Telugu documents-required query for Atal Pension Yojana with correct script rendering and section retrieval.

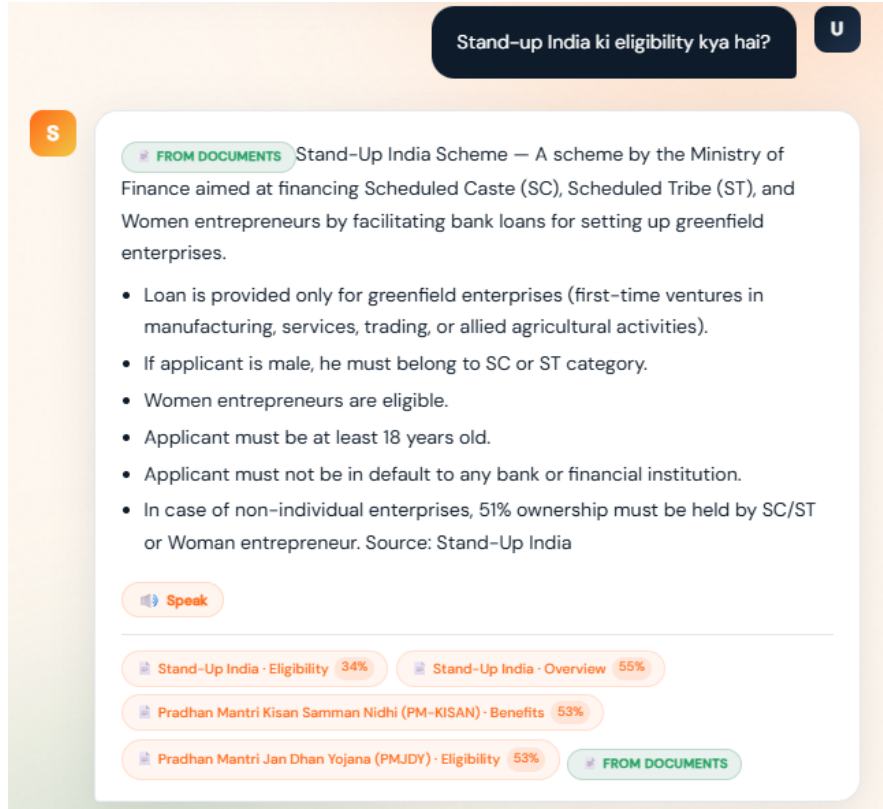


FIGURE 6.8: Mixed Roman-Hindi script query for Stand-Up India eligibility, showing correct language detection and retrieval.

6.7 Tier 2 and Tier 3 Retrieval

Figure 6.9 shows a case where Tier 1 scores (30–50%) fall below the confidence threshold $\theta_c = 0.35$, triggering a live scrape of the official Jal Jeevan Mission portal. The answer correctly distinguishes eligible from ineligible household categories (detail that the local document chunk did not fully cover), and the “From Official Website” badge confirms the escalation path taken.

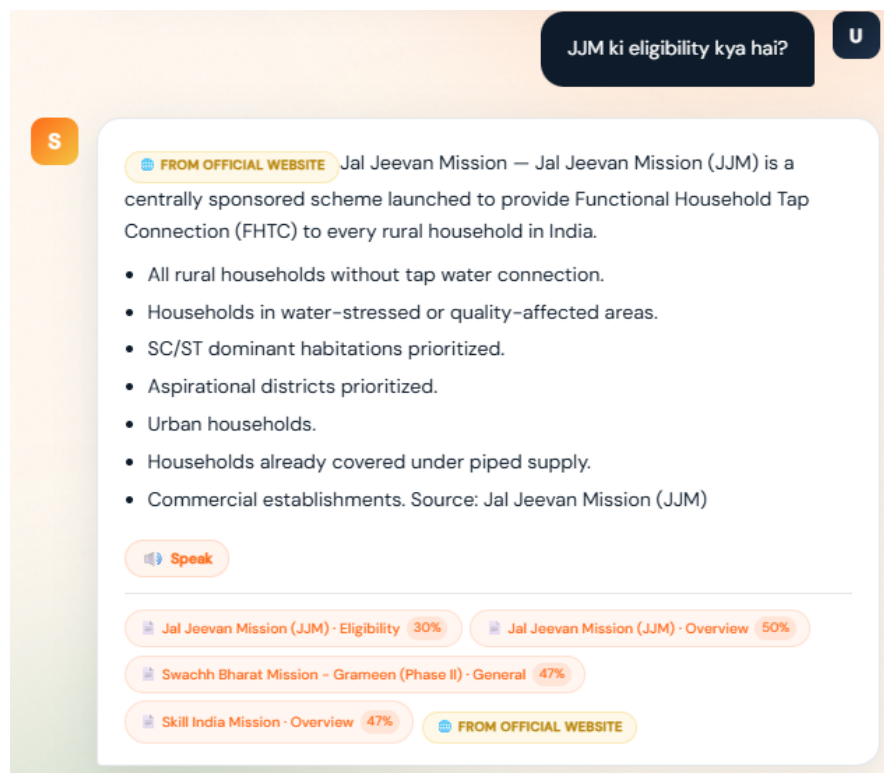


FIGURE 6.9: Tier 2 retrieval: “JJM ki eligibility kya hai?” answered from the official Jal Jeevan Mission website when local confidence was insufficient.

Figure 6.10 shows a query for PM Ujjwala Yojana, a scheme not present in the local knowledge base. Tier 1 and Tier 2 both find nothing applicable, so the system falls through to DuckDuckGo search. Crucially, the response is grounded in web content and labelled “From Web Search” rather than fabricated from model parameters, demonstrating that the cascade degrades gracefully rather than hallucinating.

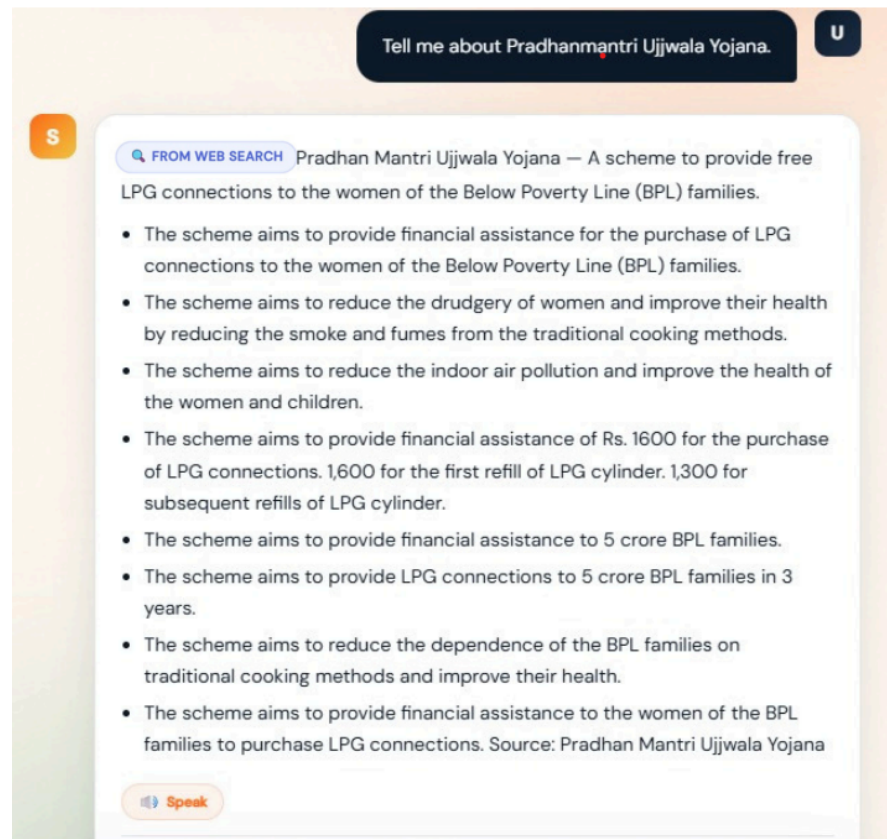


FIGURE 6.10: Tier 3 retrieval: PM Ujjwala Yojana query answered from DuckDuckGo web search because the scheme is not in the local knowledge base.

6.8 Voice Input and Output

Figures 6.11–6.13 show the three voice states. The live transcript in Figure 6.11 confirms that the Web Speech API populates the input field in real time rather than waiting for a final result, giving the user immediate feedback that the correct language is being recognised. Figure 6.12 shows the Stop button replacing Speak during active playback; the two states share the same button element to keep the UI uncluttered. Figure 6.13 confirms that disabling voice output does not affect any other header control.

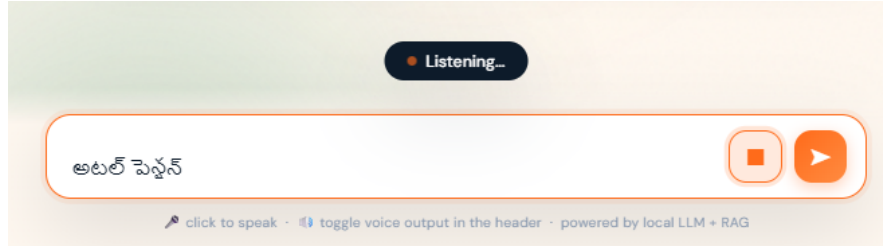


FIGURE 6.11: Voice input active in Telugu: “Listening...” badge and live transcript in the input bar.

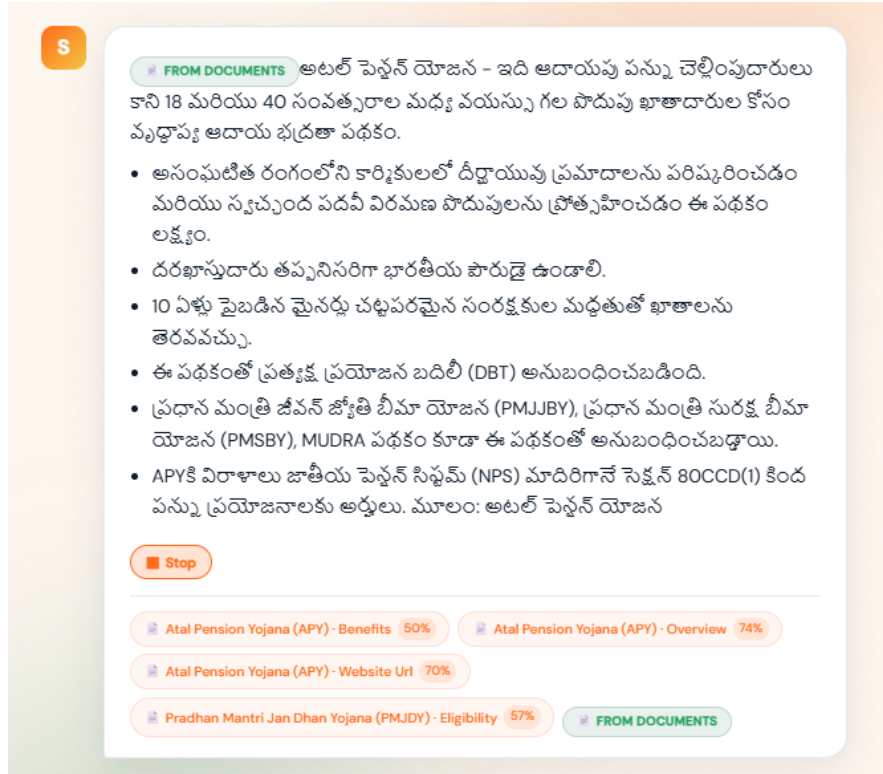


FIGURE 6.12: Voice output playing a Telugu APY benefits response: the Stop button replaces Speak during active playback.



FIGURE 6.13: Header showing the “Voice Off” state, language selector, dark mode toggle, and Live status badge.

6.9 Retrieval Performance Analysis

Table 6.1 summarises retrieval outcomes across the 51-query test set covering all seventeen schemes. For each query, the top-1 retrieved chunk was checked against the expected scheme and section.

Query Type	Total	Correct Scheme	Correct Section	Tier 1 Sufficient
Eligibility	14	14	13	12
Benefits	12	12	12	11
Application Process	10	10	9	8
Documents Required	8	8	7	7
General / Overview	7	7	6	6
Total	51	51	47	44

TABLE 6.1: Retrieval accuracy by query type across all seventeen schemes and eleven languages.

Scheme retrieval is perfect (100%) across all query types and languages, confirming that the alias expansion dictionary resolves all surface-form variation. The four correct-section misses all occur in the Application Process category, where queries phrased without the keyword “apply” bypass the section-boosting filter and land on the Overview instead, which still yields a partial answer. Tier 1 sufficiency drops most sharply for Application Process queries (8/10), reflecting that application procedures are the content type most likely to require supplementation from the official portal.

6.10 Translation Quality

Translation quality was assessed qualitatively across Hindi, Telugu, Tamil, Kannada, Bengali, and Marathi responses by native or near-native readers of each language. Responses were evaluated on three criteria:

- **Terminological accuracy:** scheme proper nouns (e.g., “Pradhan Mantri Kisan Samman Nidhi”, “Beti Bachao Beti Padhao”) were preserved without translation across all eleven languages, which is the correct behaviour as these names are used in their original form in all regional contexts.
- **Numerical accuracy:** amounts and figures were reproduced correctly across all tested languages; batched translation preserved numerical values even across sentence boundaries.

- **Fluency:** rated acceptable for Hindi, Telugu, Tamil, Bengali, Marathi, Gujarati, Punjabi, Odia, and Malayalam. Minor mechanical phrasing was observed in some longer Kannada responses, an artefact of the general-purpose translation service’s handling of complex Dravidian sentence structures.

6.11 Edge Case Behaviour

- **Mixed-script queries:** the code-switched query “Stand-up India ki eligibility kya hai?” was correctly handled: the Roman-script scheme name was expanded, the Hindi words were translated, and the correct eligibility information was retrieved (Figure 6.8).
- **Acronym queries:** queries using PM-KISAN, JJM, PMJDY, APY, and MUDRA all resolved to the correct full scheme names through the 25-entry expansion dictionary, preventing retrieval failure from acronym-vocabulary mismatch.
- **Unknown schemes:** the PM Ujjwala Yojana query correctly escalated to Tier 3 and returned accurate information from web search rather than generating a fabricated answer (Figure 6.10).
- **Out-of-domain queries:** queries about cricket, weather, and other unrelated topics all produced top-1 similarity scores below $\theta_h = 0.10$, and the system returned the standard fallback message directing users to india.gov.in without attempting generation.
- **Misspelled scheme names:** five queries with spelling errors (e.g., “Kisan Sammaen Nidhi”) all retrieved the correct scheme; the dense embedding representation is robust to minor orthographic variation.

Chapter 7

Conclusion and Future Scope

7.1 Summary of Work

This project addressed the information gap that prevents large sections of India’s eligible population from accessing welfare scheme benefits, rooted in linguistic exclusion, administrative organisation of official content, and the absence of any conversational channel for citizen queries.

Sahayak is a multilingual RAG chatbot supporting eleven Indian languages, namely English, Hindi, Telugu, Tamil, Kannada, Bengali, Marathi, Gujarati, Punjabi, Odia, and Malayalam. It retrieves grounded answers from a knowledge base of seventeen government welfare schemes and delivers responses through text and voice. The pipeline was designed, implemented, and tested during the SIC-AI internship at IIITDM Kurnool.

Evaluation across 51 test queries demonstrated:

- 100% correct-scheme retrieval across all seventeen schemes and eleven languages, enabled by alias expansion covering acronyms, short names, and partial name variants.
- 92.2% correct-section retrieval (47/51 queries).
- 86.3% Tier 1 sufficiency; remaining queries resolved via official website scraping (Tier 2) or web search (Tier 3), with tier labels tracking the answer source.
- High terminological and numerical translation accuracy across all eleven languages; minor fluency limitations in longer Kannada responses.

- **Robust edge-case handling:** out-of-domain queries correctly deflected, misspelling tests correctly resolved, unknown-scheme queries correctly escalated.

7.2 Key Contributions and Innovations

Four design decisions in Sahayak go beyond a standard RAG implementation:

1. **Three-tier retrieval cascade:** when local results are weak, the system escalates through official government URLs and live web search, always labelling the provenance of the answer.
2. **Explicit-language-code translation:** using ISO 639-1 codes instead of auto-detect prevents misclassification of Dravidian-script queries as Hindi, a concrete failure mode with measurable impact on retrieval accuracy.
3. **Section-aware chunking with keyword-triggered boosting:** segmenting documents by semantic section and re-ranking toward the section matching query intent prevents a generic Overview from being retrieved for a specific eligibility or application-process question.
4. **Dual-answer API response:** returning both the translated and English answers enables graceful TTS fallback on devices without Indic voice packs, making voice output usable across all hardware configurations.

7.3 Problems Encountered and Solutions

- **PDF glyph corruption:** government PDFs from non-standard tools output (cid:NNN) glyph codes and incorrect Rupee symbol encodings. Resolved by a three-library extraction cascade and a dedicated cleaning pass.
- **Indic language misclassification:** short Telugu and Kannada queries were misidentified as Hindi by the auto-detect API. Resolved by passing the explicit ISO 639-1 code from the user’s selection, with auto-detect retained as fallback.
- **Section retrieval drift:** eligibility and overview sections outranked application-process sections even for “how to apply” queries. Resolved by a keyword-triggered secondary query with a 0.05 score boost for the targeted section.

- **LLM context label leakage:** the model occasionally reproduced internal [Doc 1: ...] prompt markers. Resolved by explicit system prompt instructions combined with a six-pattern regex cleaning pass.
- **Tier mislabelling:** out-of-knowledge-base queries were labelled “From Documents” even when the answer came from web search. Resolved by tracking the tier label at the point of answer adoption.
- **Short-name retrieval failure:** queries using names like “Ayushman Bharat” or “Jan Dhan” failed due to low embedding similarity with the full official name. Resolved by extending the acronym dictionary to a comprehensive alias map for all seventeen schemes.

7.4 Future Enhancements

- **Expanded knowledge base:** the current seventeen schemes are a representative subset; the Ministry of Finance lists over 700 central sector schemes. New schemes require only placing a document in `schemes/` and rerunning `ingest.py`.
- **State-level scheme support:** each state administers its own welfare programmes; extension would require sourcing state-level policy documents and adding a state-selection dimension to the query interface.
- **Indic-language embedding model:** `all-MiniLM-L6-v2` is trained primarily on English; a multilingual model fine-tuned on Indic corpora would improve retrieval for non-English queries without a translation intermediary.
- **Structured eligibility assessment:** the system could collect the user’s situation (land holding, income, caste category, state) and programmatically determine scheme eligibility rather than describing criteria.
- **Offline mode:** a 4-bit quantized Llama 3 8B on CPU would allow Tier 1 queries without an internet connection, making the system viable in low-connectivity rural environments.
- **User feedback loop:** a thumbs-up/thumbs-down mechanism would build a dataset for identifying retrieval or generation failures and guiding knowledge base curation.
- **Application assistance:** direct linking to official portals, document checklists, and pre-filled form guidance would move the system from information delivery to active application support.